

Event Management System

Hadi Muselmani (202304319)

Ali kefel(202403197)

Hussain Dakik(202402938)

Event Management System – Final Project Report.....	1
1. Introduction.....	1
2. Project Objectives.....	1
3. Database Design.....	1
3.1 ER/EER Diagram.....	2
3.2 Relational Schema.....	2
4. Query Execution.....	3
1. List all events with their venue names.....	3
2. Find all sessions for a given event.....	3
3. List ticket types for each event with prices.....	3
4. Count how many tickets were purchased for each event.....	4
5. Total payment collected per event.....	4
6. Attendees with unpaid payments.....	4
7. Sponsors supporting more than one event.....	4
8. Events organized by a specific organizer.....	4
9. Speakers at a specific event.....	4
10. Attendees who bought VIP tickets.....	4
11. Event with the highest number of tickets sold.....	4
12. Events with no sponsors.....	4
13. Attendees who have not purchased any ticket.....	4
14. Ticket type generating the most revenue.....	5
15. Sessions between 10:00 and 14:00.....	5
5. Functional Dependencies.....	5
6. Relational Algebra Expressions.....	5

1. Introduction

This report presents a fully implemented Event Management System designed as part of a comprehensive database project. The system models core real-world event operations including venue scheduling, organizer coordination, session management, speaker participation, ticket sales, attendee records, and sponsor involvement. The goal is to demonstrate mastery of conceptual design, logical modeling, SQL implementation, query formulation, and relational reasoning.

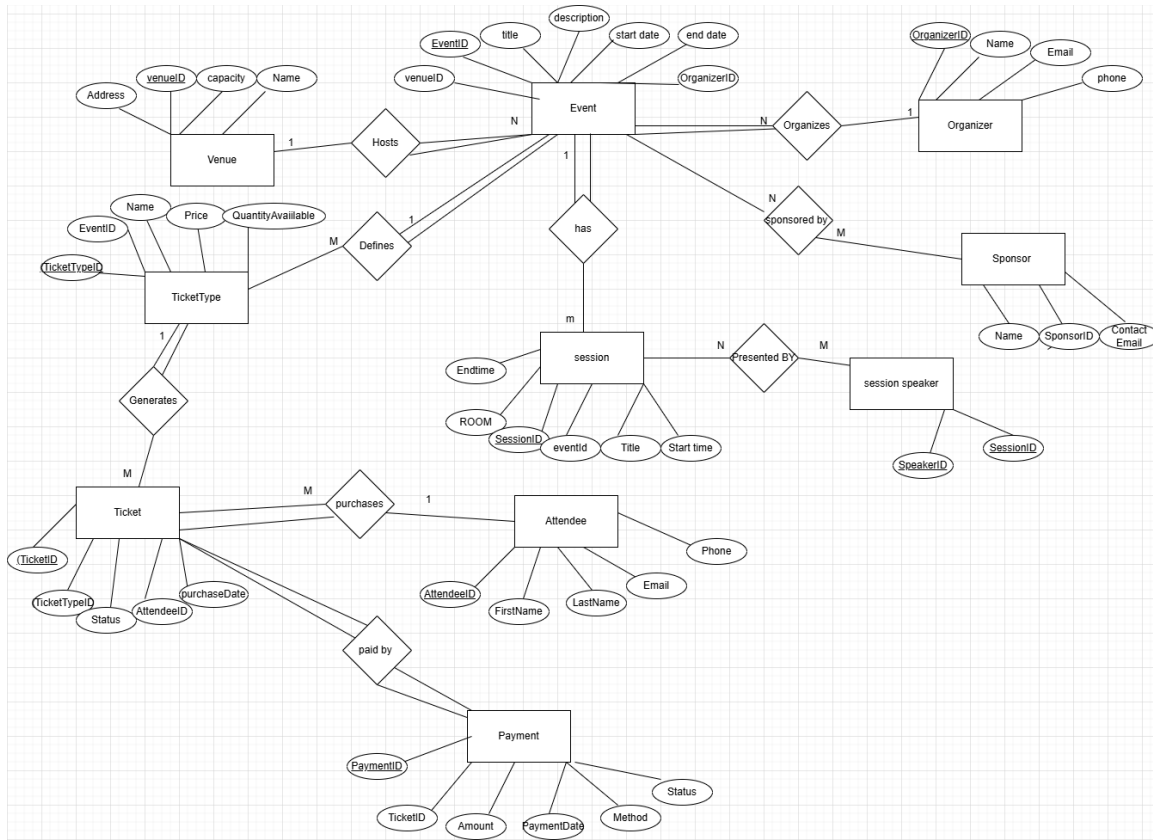
2. Project Objectives

The primary objectives of this project include:

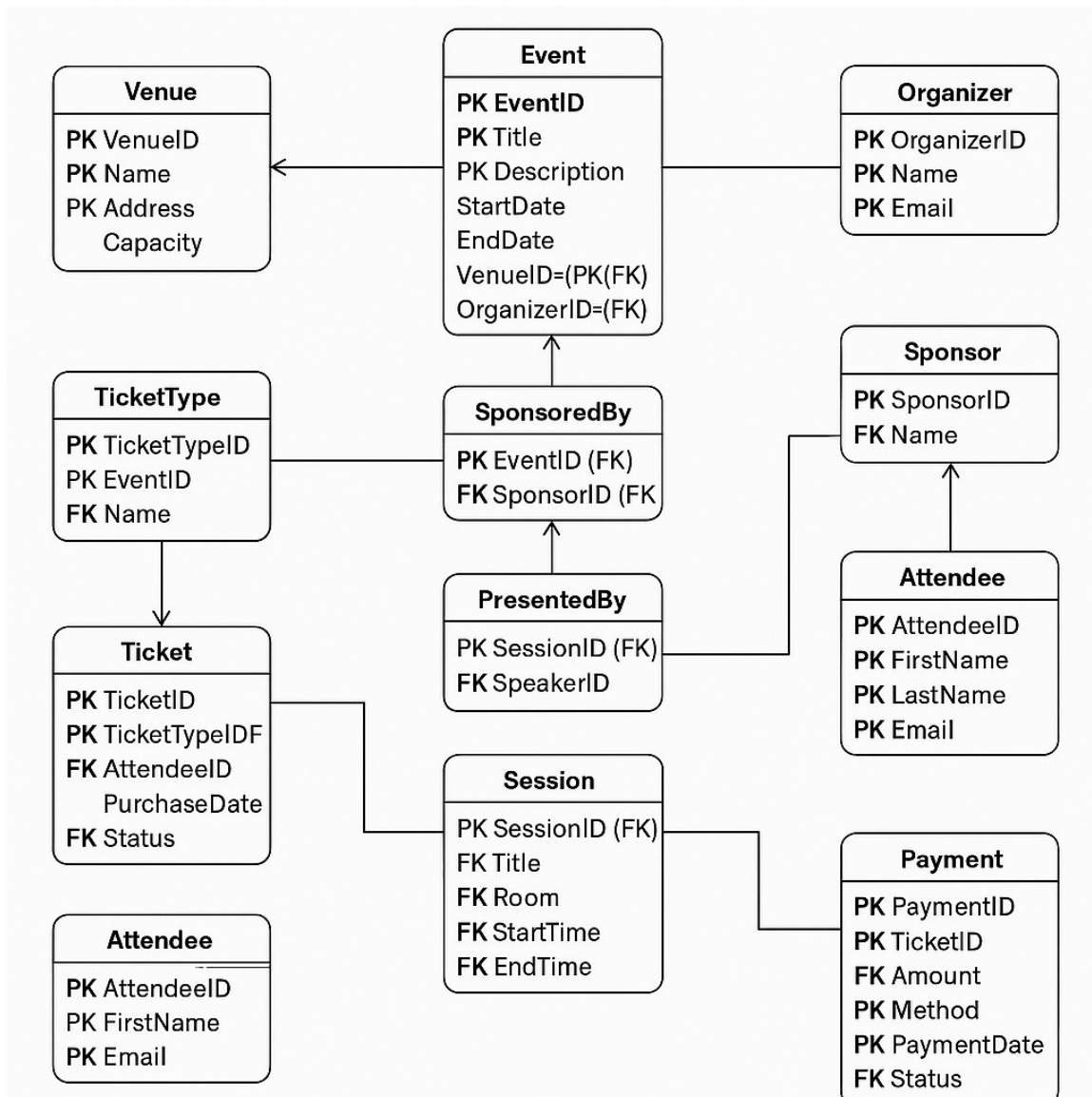
- Design a complete ER/EER model representing the event lifecycle.
- Construct a normalized relational schema with appropriate constraints.
- Implement the schema using SQL and populate it with meaningful sample data.
- Develop and execute at least fifteen diverse SQL queries demonstrating functionality.
- Provide functional dependencies and relational algebra expressions for selected queries.
- Deliver a polished and professional technical document detailing the full system.

3. Database Design

3.1 ER/EER Diagram



3.2 Relational Schema



4. Query Execution

This section includes fifteen SQL queries designed to demonstrate system capabilities. Each query is accompanied by an explanation of its purpose and logic. Outputs can be inserted as screenshots where required.

1. List all events with their venue names

```
SELECT Event.Title, Venue.Name AS VenueName FROM Event JOIN Venue ON Event.VenueID = Venue.VenueID;
```

2. Find all sessions for a given event

```
SELECT Title, StartTime, EndTime FROM Session WHERE EventID = 5;
```

3. List ticket types for each event with prices

```
SELECT Event.Title, TicketType.Name, TicketType.Price FROM TicketType JOIN Event ON  
TicketType.EventID = Event.EventID;
```

4. Count how many tickets were purchased for each event

```
SELECT Event.Title, COUNT(Ticket.TicketID) AS TicketsSold FROM Ticket JOIN TicketType ON  
Ticket.TicketTypeID = TicketType.TicketTypeID JOIN Event ON TicketType.EventID =  
Event.EventID GROUP BY Event.Title;
```

5. Total payment collected per event

```
SELECT Event.Title, SUM(Payment.Amount) AS TotalRevenue FROM Payment JOIN Ticket ON  
Payment.TicketID = Ticket.TicketID JOIN TicketType ON Ticket.TicketTypeID =  
TicketType.TicketTypeID JOIN Event ON TicketType.EventID = Event.EventID GROUP BY  
Event.Title;
```

6. Attendees with unpaid payments

```
SELECT A.FirstName, A.LastName, T.TicketID FROM Attendee A JOIN Ticket T ON A.AttendeeID  
= T.AttendeeID LEFT JOIN Payment P ON T.TicketID = P.TicketID WHERE P.Status = 'Unpaid';
```

7. Sponsors supporting more than one event

```
SELECT Sponsor.Name, COUNT(*) AS SponsoredEvents FROM SponsoredBy JOIN Sponsor ON  
SponsoredBy.SponsorID = Sponsor.SponsorID GROUP BY Sponsor.Name HAVING COUNT(*) > 1;
```

8. Events organized by a specific organizer

```
SELECT Title, StartDate, EndDate FROM Event WHERE OrganizerID = 3;
```

9. Speakers at a specific event

```
SELECT Speaker.SpeakerID, Speaker.Name FROM Session JOIN PresentedBy ON  
Session.SessionID = PresentedBy.SessionID JOIN SessionSpeaker AS Speaker ON  
PresentedBy.SpeakerID = Speaker.SpeakerID WHERE Session.EventID = 2;
```

10. Attendees who bought VIP tickets

```
SELECT FirstName, LastName FROM Attendee WHERE AttendeeID IN (SELECT AttendeeID  
FROM Ticket WHERE TicketTypeID IN (SELECT TicketTypeID FROM TicketType WHERE  
Name='VIP'));
```

11. Event with the highest number of tickets sold

```
SELECT TOP 1 Event.Title, COUNT(TicketID) AS TicketsSold FROM Ticket JOIN TicketType ON  
Ticket.TicketTypeID = TicketType.TicketTypeID JOIN Event ON TicketType.EventID =  
Event.EventID GROUP BY Event.Title ORDER BY TicketsSold DESC;
```

12. Events with no sponsors

```
SELECT Title FROM Event E WHERE NOT EXISTS (SELECT 1 FROM SponsoredBy S WHERE  
S.EventID = E.EventID);
```

13. Attendees who have not purchased any ticket

```
SELECT A.FirstName, A.LastName FROM Attendee A WHERE NOT EXISTS (SELECT 1 FROM Ticket T WHERE T.AttendeeID = A.AttendeeID);
```

14. Ticket type generating the most revenue

```
SELECT TOP 1 TicketType.Name, SUM(Payment.Amount) AS Revenue FROM Payment JOIN Ticket ON Payment.TicketID = Ticket.TicketID JOIN TicketType ON Ticket.TicketTypeID = TicketType.TicketTypeID GROUP BY TicketType.Name ORDER BY Revenue DESC;
```

15. Sessions between 10:00 and 14:00

```
SELECT Title, StartTime, EndTime FROM Session WHERE StartTime BETWEEN '10:00:00' AND '14:00:00'
```

16. Events sponsored by a specific sponsor

```
SELECT E.EventID, E.Title, E.StartDate, E.EndDate FROM Event E JOIN SponsoredBy SB ON E.EventID = SB.EventID WHERE SB.SponsorID = 1;
```

17. List all events taking place at a specific venue

```
SELECT E.EventID, E.Title AS EventTitle, E.StartDate, V.Name AS VenueName FROM Event E JOIN Venue V ON E.VenueID = V.VenueID WHERE V.Name = 'Grand Hall';
```

18. List all speakers who are participating in more than one session

```
SELECT S.SpeakerID, S.Name AS SpeakerName, COUNT(P.SessionID) AS TotalSessions FROM SessionSpeaker S JOIN PresentedBy P ON S.SpeakerID = P.SpeakerID GROUP BY S.SpeakerID, S.Name HAVING COUNT(P.SessionID) > 1;
```

19. Ticket types and their total revenue per event

```
SELECT E.EventID, E.Title AS EventTitle, TT.TicketTypeID, TT.Name AS TicketType, SUM(P.Amount) AS TotalRevenue FROM TicketType TT JOIN Ticket T ON TT.TicketTypeID = T.TicketTypeID JOIN Payment P ON T.TicketID = P.TicketID JOIN Event E ON TT.EventID = E.EventID GROUP BY E.EventID, E.Title, TT.TicketTypeID, TT.Name ORDER BY TotalRevenue DESC;
```

20. Events that have no sessions assigned yet

```
SELECT E.EventID, E.Title AS EventTitle FROM Event E LEFT JOIN Session S ON E.EventID = S.EventID WHERE S.SessionID IS NULL;
```

5. Functional Dependencies

This section lists the functional dependencies for each relation in the database. These dependencies confirm that the design is normalized and each table maintains a single theme.

- EventID $\rightarrow$ Title, Description, StartDate, EndDate, VenueID, OrganizerID
- VenueID $\rightarrow$ Name, Address, Capacity
- OrganizerID $\rightarrow$ Name, Email, Phone
- SponsorID $\rightarrow$ Name, ContactEmail
- TicketTypeID $\rightarrow$ EventID, Name, Price, QuantityAvailable
- SessionID $\rightarrow$ EventID, Title, Room, StartTime, EndTime
- SpeakerID $\rightarrow$ Name, ContactEmail
- TicketID $\rightarrow$ TicketTypeID, AttendeeID, PurchaseDate, Status
- AttendeeID $\rightarrow$ FirstName, LastName, Email, Phone
- PaymentID $\rightarrow$ TicketID, Amount, Method, PaymentDate, Status

6. Relational Algebra Expressions

relational algebra expressions used in this project:

Query 1 — List all events with their venue names

$\pi_{\text{Title, Name}} (\text{Event} \bowtie_{\text{Event.VenueID=Venue.VenueID}} \text{Venue})$

Query 2 — Retrieve all sessions for EventID = 1

$\pi_{\text{Title, StartTime, EndTime}} (\sigma_{\text{EventID=1}} (\text{Session}))$

Query 3 — Display ticket types for each event with prices

$\pi_{\text{Title, Name, Price}} (\text{Event} \bowtie_{\text{Event.EventID=TicketType.EventID}} \text{TicketType})$

Query 12 — Events with NO sponsors

$\text{Event} - \pi_{\text{EventID}} (\text{SponsoredBy})$

Query 13 — Attendees who have NOT purchased any ticket

$\text{Attendee} - \pi_{\text{AttendeeID}} (\text{Ticket})$

7. Screenshots & Sample Data